



School of Future Tech

Case Study Report

on

Digital Content Publishing Router

by

Arnesh Sunil Munj

150096725034

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.

5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

In modern software engineering, the reliance on third-party APIs (such as Google Maps or Mapbox) for geospatial logic creates massive financial overhead and dependency bottlenecks. This case study examines the development of the "Geolocation Travel Distance Tracking Playground", an advanced frontend engineering project. The project was built to demonstrate that complex spatial tracking, distance computation, route visualization, and data persistence can be engineered entirely on the client-side without relying on proprietary backend mapping servers or external database dependencies. By leveraging the device's native hardware, the project establishes a sovereign, privacy-first analytics platform.

2. Problem Statement / Case Background (Abstract)

Background

Standard geospatial tracking applications are tightly coupled with proprietary mapping APIs. This results in heavy data payloads, lack of offline functionality, and high recurring costs. Furthermore, raw GPS data provided by consumer mobile hardware is inherently noisy. Without strict mathematical filtering, satellite interference causes "route spiking," which leads to wildly inaccurate distance accumulation (e.g., logging movement even when the user is standing still).

Abstract

This project implements a lightweight, browser-based geotracking solution using React JS and the native HTML5 Geolocation API. By manually implementing the Haversine formula, developing a real-time coordinate filtering pipeline, and utilizing SVG-based relative mapping, the application achieves enterprise-grade tracking accuracy entirely offline. The result is a robust, isolated telemetry platform capable of monitoring speed, bearing, and distance while exporting standardized GIS data formats (GPX, CSV, JSON).

3. Case Study Design

The system was designed with a strict Unidirectional Data Flow pattern controlled by Zustand to ensure the UI does not freeze under the heavy load of constant GPS tracking.

Data Acquisition Layer: Interfaces directly with the device hardware via the HTML5 Geolocation API (`navigator.geolocation.watchPosition`).

Validation Layer: An algorithmic gatekeeper that intercepts the raw payload and rejects any coordinate with an accuracy radius worse than 30 meters.

Controller/Math Layer: Computes real-time metrics (Distance, Velocity, Compass Bearing) using spherical trigonometry and updates the global state.

View Layer (React): Subscribes to the global state and renders UI components, including the Odometer Scoreboard and the dynamic SVG Map Canvas.

Persistence Layer: Serializes completed JSON state into the browser's LocalStorage upon journey termination.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

To ensure mathematical accuracy on a spherical planet, the project avoids flat 2D Cartesian math (Pythagorean theorem) and relies on rigorous geospatial algorithms:

1. The Haversine Formula: Used to compute the great-circle distance between two points over the Earth's curvature.

Formula: $a = \sin^2(\Delta\phi/2) + \cos \phi_1 \cdot \cos \phi_2 \cdot \sin^2(\Delta\lambda/2)$

Distance: $d = R \cdot (2 \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a}))$ (where R is Earth's radius: 6371 km).

2. Bearing Algorithm: Used to determine travel direction (e.g., North, South-East) by calculating the initial compass bearing relative to True North between the previous and current coordinates.

Formula: $\theta = \text{atan2}(\sin \Delta\lambda \cdot \cos \phi_2, \cos \phi_1 \cdot \sin \phi_2 - \sin \phi_1 \cdot \cos \phi_2 \cdot \cos \Delta\lambda)$

3. SVG Bounding Box Interpolation: Used to draw the route map offline. The algorithm finds the min/max latitude and longitude of the coordinate array and uses linear interpolation to translate geographic coordinates into relative X/Y pixel space on an SVG canvas.

5. Problem Statement / Case Study Implementation Details

The core implementation relies on React JS (v18+) for the component architecture and Zustand for avoiding deep prop-drilling and unnecessary re-renders.

The GPS Engine: Configured to request maximum hardware precision using `{ enableHighAccuracy: true, maximumAge: 0, timeout: 10000 }`.

Export Pipeline: To ensure data portability without a backend server, the Javascript Blob API is used. The application translates the internal JSON state into strict XML-compliant .gpx schemas or comma-separated .csv strings, allowing users to download their routes directly to their local machine.

Developer Simulation Mode: A unique implementation that bypasses the hardware API entirely. It uses trigonometry to generate synthetic coordinate geometry (Square, Circular, and Zig-Zag routes) and injects them directly into the state manager to test the application indoors.

(Note: For the "Snapshots" portion of your submission, insert the screenshots gathered from your Dashboard, Landing Page, and Route Simulator here).

6. Problem Statement / Case Study Results and Conclusion.

Results: The finalized application successfully operates 100% offline. When tested against physical devices, the implemented 30-meter accuracy filter successfully mitigated the "GPS Drift" phenomenon, resulting in highly accurate distance accumulation. The SVG mapping engine successfully renders massive coordinate arrays smoothly without memory exhaustion due to React useMemo optimizations. The Blob export engine successfully constructs raw coordinates into interoperable GPX schemas ready for external GIS software.

Conclusion: The Geolocation Tracking Playground proves that robust geospatial telemetry platforms do not strictly require heavy cloud infrastructure or proprietary map APIs. By combining raw hardware APIs, rigorous mathematical filters, and highly optimized React state management, developers can achieve powerful offline analytics tools that are performant, privacy-preserving, and highly portable.

7. References

- Mozilla Developer Network (MDN) Web Docs: Geolocation API & navigator.geolocation.
- Sinnott, R. W. (1984): Virtues of the Haversine. Sky and Telescope.
- Zustand Documentation: Bear Necessities for State Management in React.
- React JS Official Documentation: Hooks, Component Lifecycle, and useMemo Optimizations.

- Topografix (2004): The GPX 1.1 Schema Documentation.